

✓ 7-Class Emotion Recognition: ResNet and ConvNeXt CNN Architectures

Introduction

This project explores the application of deep convolutional neural networks (CNNs) for recognizing emotions in seven classes. We will utilize two well-known architectures based on residual blocks: [ResNet](#) and [ConvNeXt](#). Given their architectural similarities, our goal is to compare their performance and identify the optimal model for classifying facial expressions into the following emotional states:

- Angry
- Disgust
- Fear
- Happy
- Sad
- Surprise
- Neutral

We will be using the [FER-2013](#) dataset from Kaggle, a collection of 48x48 pixel grayscale images of faces, each labeled with one of the seven emotions mentioned above. We will evaluate our models using accuracy, precision, recall, and F1 score, with a particular focus on the F1 score as our primary metric for comparison.

✓ Importing libraries

```

!pip install -q datasets
!pip install -q transformers[torch]
!pip install -q accelerate -U
!pip install -q torchsampler

import torch
import torchvision
from torchvision import datasets, transforms, models
from torch.utils.data import random_split, DataLoader
import matplotlib.pyplot as plt
import numpy as np
from huggingface_hub import notebook_login
import matplotlib.pyplot as plt
import seaborn as sns
import torch.nn as nn
import random
from transformers import (
    get_linear_schedule_with_warmup,
    EarlyStoppingCallback
)
from torch.optim import AdamW
from torch.optim.lr_scheduler import ReduceLROnPlateau
from torch.nn import CrossEntropyLoss # For loss calculation
from tqdm.auto import tqdm
from sklearn.metrics import accuracy_score, precision_score, f1_score, confusion_matrix, roc_auc_score, classification_report, precision_recall_fscore_support
from transformers import EarlyStoppingCallback
from torch.optim.lr_scheduler import StepLR
from sklearn.preprocessing import LabelBinarizer

def set_seed(seed_value):
    """Set seed for reproducibility."""
    random.seed(seed_value)
    np.random.seed(seed_value)
    torch.manual_seed(seed_value)
    torch.cuda.manual_seed_all(seed_value)

```

```

notebook_login()
#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

```

 [Show hidden output](#)

Loading and Splitting the Dataset from HuggingFace

```

from datasets import load_dataset, DatasetDict

set_seed(42)
dataset = load_dataset("Luuu01/probaneka")
labels = dataset["train"].features["label"].names
id2label = {k:v for k,v in enumerate(labels)}
label2id = {v:k for k,v in enumerate(labels)}
train_test_dataset = dataset['train'].train_test_split(test_size=0.2, seed=42) # 80/20

# Reorganize
dataset = DatasetDict({
    'train': train_test_dataset['train'],

```

```
'test': train_test_dataset['test'],
'validation': dataset['validation']
})

dataset

DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 23056
  })
  test: Dataset({
    features: ['image', 'label'],
    num_rows: 5765
  })
  validation: Dataset({
    features: ['image', 'label'],
    num_rows: 7066
  })
})
```

Let's visualize the first training example:

```
example = dataset["train"][0]
example["image"]
```



Check its corresponding label:

```
example['label']
```



2

Looks like the image above is labeled as 'fear':

```
id2label[2]
```



'fear'

Let's visualize the distribution of emotions in our dataset to assess any potential class imbalances. It displays the number of instances of each emotion in the training, test, and validation sets.

While there is a slight class imbalance, we anticipate that our models will be robust enough to handle this issue effectively... 🙌

```

train_labels = dataset['train']['label']
test_labels = dataset['test']['label']
validation_labels = dataset['validation']['label']

class_names = ["angry", "disgust", "fear", "happy", "neutral", "sad", "surprise"]

fig, axes = plt.subplots(1, 3, figsize=(15, 5), sharey=True)

sns.countplot(x=train_labels, ax=axes[0])
axes[0].set_title('Train Set')
axes[0].set_xticks(range(7))
axes[0].set_xticklabels(class_names, rotation=45)

sns.countplot(x=test_labels, ax=axes[1])
axes[1].set_title('Test Set')
axes[1].set_xticks(range(7))
axes[1].set_xticklabels(class_names, rotation=45)

sns.countplot(x=validation_labels, ax=axes[2])
axes[2].set_title('Validation Set')
axes[2].set_xticks(range(7))
axes[2].set_xticklabels(class_names, rotation=45)

plt.suptitle('Emotion Class Balance', y=1, fontsize=18)
plt.show()

```

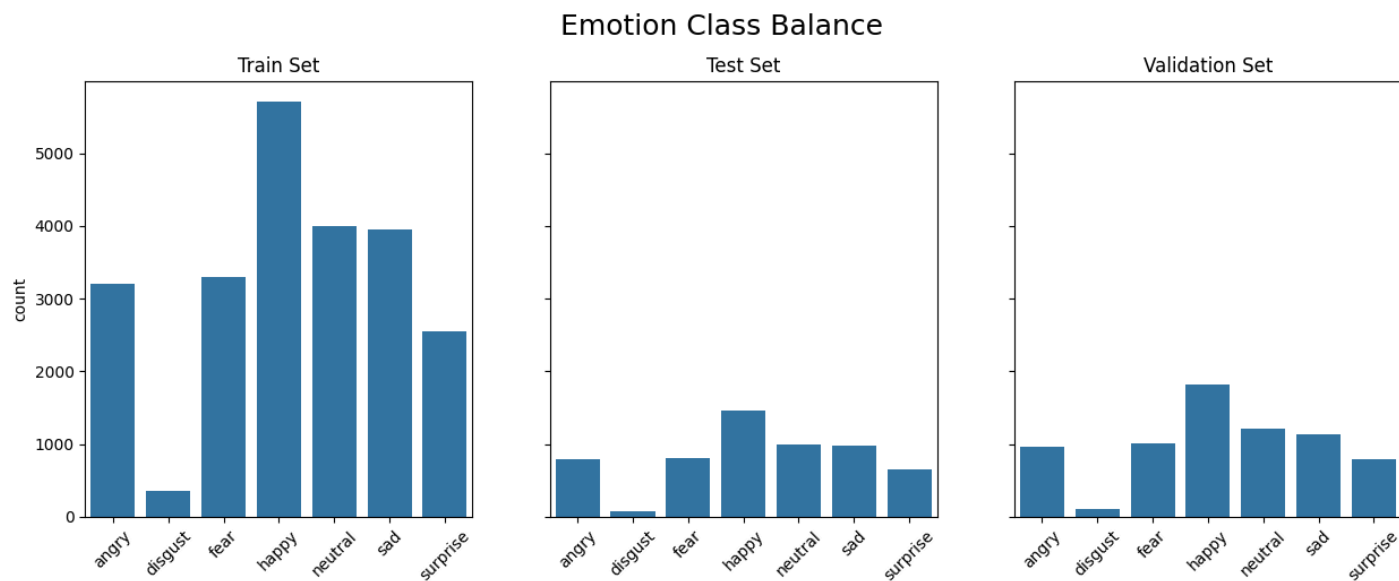


Image Preprocessing and Transformations

This section defines image transformations to prepare our data for training and evaluation.

- **Training Set:** We apply data augmentation techniques such as random horizontal flipping and color jittering to increase the diversity of the training data and improve the model's ability to generalize to unseen examples.
- **Validation and Test Sets:** For the validation and test sets, we apply only essential transformations like resizing and normalization to ensure consistency across all images.

```
from torchvision.transforms import (
    Compose,
    Normalize,
    RandomHorizontalFlip,
    ToTensor,
    Resize,
    ColorJitter
)
from transformers import AutoImageProcessor
set_seed(42)
# If you want to use pretrained image_processor and normalize for RGB images..
# image_processor = AutoImageProcessor.from_pretrained("microsoft/resnet-50")
# normalize = Normalize(mean=image_processor.image_mean, std=image_processor.image_std)

# Normalize pixel values to have a mean of 0.507 and standard deviation of 0.201.
# These values were calculated specifically for the grayscale images in our dataset.
normalize = Normalize(mean=[0.507], std=[0.201])

train_transform = Compose([
    Resize((224,224)),
    RandomHorizontalFlip(),
    ColorJitter(brightness=0.33, contrast=0.33),
    ToTensor(),
    normalize,
])

eval_transform = Compose([
    Resize((224,224)),
    ToTensor(),
    normalize,
])

def train_transforms(examples):
    examples["pixel_values"] = [train_transform(image) for image in examples["image"]]
    return examples

def eval_transforms(examples):
    examples["pixel_values"] = [eval_transform(image) for image in examples["image"]]
    return examples

# Apply transformations on-the-fly when loading data (improves memory efficiency)
train_dataset = dataset['train'].with_transform(train_transforms)
val_dataset = dataset['validation'].with_transform(eval_transforms)
test_dataset = dataset['test'].with_transform(eval_transforms)
```

✎ Create DataLoaders for Training, Validation, and Testing

This section creates PyTorch DataLoaders for the training, validation, and test datasets. These loaders efficiently handle batching, shuffling (for training), and collation of data for model training and evaluation.

```
set_seed(42)
def collate_fn(examples):
    pixel_values = torch.stack([example["pixel_values"] for example in examples])
    labels = torch.tensor([example["label"] for example in examples])
    return {"pixel_values": pixel_values, "labels": labels}

train_loader = DataLoader(train_dataset, collate_fn=collate_fn, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, collate_fn=collate_fn, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, collate_fn=collate_fn, batch_size=32, shuffle=False)
```

Let's take a quick peek at the first batch to make sure our DataLoader is working as it should.

```
batch = next(iter(train_loader))
for k,v in batch.items():
    print(k,v.shape)
```

```
🔄 pixel_values torch.Size([16, 1, 224, 224])
    labels torch.Size([16])
```

✓ Define models

Next, we'll define our pretrained ResNet and ConvNeXt models, which we'll fine-tune for our 7-class emotion recognition task on grayscale images. We'll use Hugging Face's AutoModelForImageClassification to load the models and adapt them to our specific dataset and number of classes.

```
from transformers import (
    AutoModelForImageClassification,
    TrainingArguments,
    Trainer,
)

model1 = AutoModelForImageClassification.from_pretrained(
    "facebook/convnext-tiny-224",
    num_channels = 1,
    num_labels=7,
    id2label=id2label,
    label2id=label2id,
    ignore_mismatched_sizes=True,
)

model2 = AutoModelForImageClassification.from_pretrained(
    "microsoft/resnet-50",
    num_channels = 1,
    num_labels=7,
    id2label=id2label,
    label2id=label2id,
    ignore_mismatched_sizes=True,
)
```

 [Show hidden output](#)

Optional!

If you need to adjust how the ConvNeXt model handles the positional information of your input images (e.g., when working with different image sizes), you can use this snippet as a starting point. It also includes a quick test to ensure everything is working smoothly.

```
# Adjust the input size for the positional embeddings
first_conv_layer = model1.convnext.embeddings.patch_embeddings # Access the first convolutional layer
model1.convnext.embeddings.position_embeddings = nn.Parameter(
    torch.zeros(1, 48 // first_conv_layer.stride[0] * 48 // first_conv_layer.stride[1], first_conv_layer.out_channels)
)

# Test forward pass to ensure no errors
sample_input = torch.randn(1, 1, 48, 48)
output = model1.convnext.embeddings.patch_embeddings(sample_input)
print("Output shape after the first convolutional layer:", output.shape)
```

 [Show hidden output](#)

The same can be done with ResNet too.

```
first_conv_layer = model2.resnet.embedder.embedder.convolution
model2.resnet.embedder.position_embeddings = nn.Parameter(
    torch.zeros(1, 48 // first_conv_layer.stride[0] * 48 // first_conv_layer.stride[1], first_conv_layer.out_channels)
)

# Test forward pass to ensure no errors
sample_input = torch.randn(1, 1, 48, 48)
output = model2.resnet.embedder.embedder(sample_input)
print("Output shape after the first convolutional layer:", output.shape)
```

 [Show hidden output](#)

Freezing Initial Layers for Feature Extraction

This code freezes the initial layers of the ConvNeXt and ResNet models to use them as fixed feature extractors. This allows us to focus on fine-tuning the later layers for our specific emotion recognition task. We verify that the desired layers are frozen by checking if their parameters require gradients. You can easily modify this to freeze additional layers if needed.

```
for name, param in model1.named_parameters():
    if name.startswith(("convnext.embeddings", "convnext.encoder.stages.0", "convnext.encoder.stages.1")):
        param.requires_grad = False
```

```
for name, param in model1.named_parameters():
    print(f"{name}: {param.requires_grad}")
```

 [Show hidden output](#)

We're doing the same thing with the ResNet model.

```
for name, param in model2.named_parameters():
    if 'stages.3' not in name and 'classifier' not in name and "stages.2" not in name:
        param.requires_grad = False
```

```
for name, param in model2.named_parameters():
    print(f"{name}: {param.requires_grad}")
```



Show hidden output

```
def compute_metrics(p):
    preds = p.predictions.argmax(-1)
    precision, recall, f1, _ = precision_recall_fscore_support(p.label_ids, preds, average='weighted')
    acc = accuracy_score(p.label_ids, preds)
    return {
        'accuracy': acc,
        'precision': precision,
        'recall': recall,
        'f1': f1,
    }

def collate_fn(examples):
    pixel_values = torch.stack([example["pixel_values"] for example in examples])
    labels = torch.tensor([example["label"] for example in examples])
    return {"pixel_values": pixel_values, "labels": labels}
```

✓ Train the models

This setup provides a comprehensive framework for training and evaluating an emotion recognition model using Hugging Face's tools and techniques.

1. Import Libraries: Necessary libraries and functions are imported from Hugging Face's transformers, sklearn, and torch modules.
2. Metric Computation: The compute_metrics function calculates accuracy, precision, recall, and F1 score using the predictions and true labels.
3. Data Collation: The collate_fn function stacks pixel values and labels into batches for the model.
4. Training Arguments: TrainingArguments defines the training parameters, including output directory, evaluation and save strategies, learning rate, batch sizes, number of epochs, weight decay, logging settings, and conditions for loading the best model.
5. Trainer Initialization: The Trainer class is initialized with the model, training arguments, datasets, metrics computation function, data collation function, and early stopping callback.
6. Model Training: The train method is called to train the model, and results are saved and logged.
7. Model Evaluation: The model is evaluated on a test dataset, and evaluation metrics are printed, saved, and logged.


```

set_seed(42)

warmup_ratio = 0.1
# Define training arguments
training_args = TrainingArguments(
    output_dir="./CONVNEXT",          # output directory
    eval_strategy="epoch",            # evaluation strategy
    save_strategy="epoch",            # save strategy
    learning_rate=1e-3,               # learning rate
    per_device_train_batch_size=64,    # batch size for training
    per_device_eval_batch_size=64,     # batch size for evaluation
    num_train_epochs=10,              # number of training epochs
    weight_decay=0.01,               # strength of weight decay
    logging_dir="./logs",             # directory for storing logs
    logging_steps=10,                # log every X updates steps
    save_total_limit=3,               # limit the total amount of checkpoints
    remove_unused_columns=False,      # keep the columns for image processing
    load_best_model_at_end=True,
    metric_for_best_model='f1',
)

optimizer = AdamW(model1.parameters(), lr=training_args.learning_rate)
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

trainer = Trainer(
    model1,
    training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
    data_collator = collate_fn,
    optimizers=(optimizer, scheduler),
    # callbacks=[EarlyStoppingCallback(early_stopping_patience=3, early_stopping_threshold = 0.01)]
)

train_results = trainer.train()

trainer.save_model()
trainer.log_metrics("train", train_results.metrics)
trainer.save_metrics("train", train_results.metrics)
trainer.save_state()

metrics = trainer.evaluate(eval_dataset=test_dataset)
print(f"\nTest metrics: {metrics}")

trainer.log_metrics("eval", metrics)
trainer.save_metrics("eval", metrics)

```

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	1.470400	1.414599	0.428673	0.348570	0.428673	0.361282
2	1.232000	1.190213	0.553354	0.534997	0.553354	0.517644
3	1.120800	1.069141	0.598924	0.584016	0.598924	0.569942
4	1.013200	1.052421	0.607840	0.610107	0.607840	0.592660
5	0.789500	0.955498	0.652986	0.654297	0.652986	0.649489
6	0.808400	0.954866	0.656524	0.657090	0.656524	0.655504
7	0.714700	0.964631	0.656949	0.665026	0.656949	0.659307
8	0.699900	0.962294	0.663742	0.664089	0.663742	0.661894
9	0.628900	0.965672	0.661761	0.661269	0.661761	0.660527
10	0.707200	0.967279	0.661761	0.662584	0.661761	0.660988

```
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision and F-score ar
_warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision and F-score ar
_warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision and F-score ar
_warn_prf(average, modifier, msg_start, len(result))
```

```
**** train metrics ****
epoch                =          10.0
total_flos           = 1798566829GF
train_loss           =          0.9489
train_runtime        =    0:44:37.12
train_samples_per_second =          86.122
train_steps_per_second  =          1.348
```

[91/91 00:27]

```
Test metrics: {'eval_loss': 0.972163736820221, 'eval_accuracy': 0.6520381613183001, 'eval_precision': 0.6493783542867384, 'eval_r
```

```
**** eval metrics ****
epoch                =          10.0
eval_accuracy        =          0.652
eval_f1              =          0.6491
eval_loss            =          0.9722
eval_precision       =          0.6494
eval_recall          =          0.652
eval_runtime         =    0:00:27.48
eval_samples_per_second =          209.784
eval_steps_per_second  =          3.311
```

```

set_seed(42)

warmup_ratio = 0.1
# Define training arguments
training_args = TrainingArguments(
    output_dir="./RESNETT",          # output directory
    eval_strategy="epoch",          # evaluation strategy
    save_strategy="epoch",          # save strategy
    learning_rate=1e-3,             # learning rate
    per_device_train_batch_size=64,  # batch size for training
    per_device_eval_batch_size=64,   # batch size for evaluation
    num_train_epochs=10,             # number of training epochs
    weight_decay=0.01,               # strength of weight decay
    logging_dir="./logs",            # directory for storing logs
    logging_steps=10,                # log every X updates steps
    save_total_limit=3,              # limit the total amount of checkpoints
    remove_unused_columns=False,     # keep the columns for image processing
    load_best_model_at_end=True,
    metric_for_best_model='f1',
)

optimizer = AdamW(model1.parameters(), lr=training_args.learning_rate)
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

trainer = Trainer(
    model2,
    training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
    data_collator = collate_fn,
    optimizers=(optimizer, scheduler),
    # callbacks=[EarlyStoppingCallback(early_stopping_patience=3, early_stopping_threshold = 0.01)]
)

train_results = trainer.train()

trainer.save_model()
trainer.log_metrics("train", train_results.metrics)
trainer.save_metrics("train", train_results.metrics)
trainer.save_state()

metrics = trainer.evaluate(eval_dataset=test_dataset)
print(f"\nTest metrics: {metrics}")

trainer.log_metrics("eval", metrics)
trainer.save_metrics("eval", metrics)

#not working somehow..

```



Show hidden output

This script trains and evaluates a multi-class emotion recognition model using PyTorch to compare with a previous training run using Hugging Face's Trainer. It includes:

1. ROC AUC Score:

- Computes multi-class ROC AUC scores using binarized labels and predictions.

2. Model Evaluation:

- Calculates accuracy, precision, ROC AUC, F1 score, and confusion matrix.
- Computes average loss over the dataset.

3. Model Training:

- Utilizes CrossEntropyLoss and AdamW optimizer with learning rate scheduling.
- Includes forward and backward propagation, weight updates, and loss computation.
- Tracks training and validation metrics, with early stopping based on validation accuracy.

4. Visualization:

- Generates plots for training and validation loss and accuracy curves.

5. Testing:

- Evaluates the best model on the test dataset, printing test loss and metrics.

6. Device Configuration:

- Uses GPU if available for efficient computation.

7. Reproducibility:

- Sets a random seed for consistent results

This approach provides fine-grained control over the training process, enabling detailed performance analysis and tuning.

```

def multiclass_roc_auc_score(y_test, y_pred, average="macro"):
    lb = LabelBinarizer()
    lb.fit(y_test)
    y_test = lb.transform(y_test)
    y_pred = lb.transform(y_pred)
    return roc_auc_score(y_test, y_pred, average=average)

def evaluate(model, data_loader, criterion, device):
    model.eval()
    all_preds, all_labels = [], []
    total_loss = 0.0
    with torch.no_grad():
        for batch in tqdm(data_loader, desc="Evaluating"):
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(pixel_values=batch["pixel_values"])
            loss = criterion(outputs.logits, batch["labels"])
            total_loss += loss.item()
            preds = outputs.logits.argmax(-1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(batch["labels"].cpu().numpy())

    avg_loss = total_loss / len(data_loader)
    metrics = {
        'accuracy': accuracy_score(all_labels, all_preds),
        'precision': precision_score(all_labels, all_preds, average='weighted'),
        'roc_auc': multiclass_roc_auc_score(all_labels, all_preds),
        'f1': f1_score(all_labels, all_preds, average='weighted'),
        'confusion_matrix': confusion_matrix(all_labels, all_preds),
    }

    return avg_loss, metrics

def train_model(model, train_loader, val_loader, test_loader, device, num_epochs=5, lr=1e-3, weight_decay=0.01, step_size=5, gamma=0.5, patience=3):
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
    scheduler = StepLR(optimizer, step_size=step_size, gamma=gamma)

    model.to(device)
    train_losses, val_losses = [], []
    train_accuracies, val_accuracies = [], []
    best_val_accuracy = 0.0
    best_model_state = None
    epochs_no_improve = 0

    for epoch in range(num_epochs):
        model.train()
        total_train_loss = 0.0
        all_train_preds, all_train_labels = [], []
        for batch in tqdm(train_loader, desc=f"Epoch {epoch + 1}/{num_epochs}"):
            batch = {k: v.to(device) for k, v in batch.items()}
            optimizer.zero_grad()
            outputs = model(pixel_values=batch["pixel_values"])
            loss = criterion(outputs.logits, batch["labels"])
            total_train_loss += loss.item()
            loss.backward()
            optimizer.step()
            preds = outputs.logits.argmax(-1)
            all_train_preds.extend(preds.cpu().numpy())
            all_train_labels.extend(batch["labels"].cpu().numpy())

```

```

avg_train_loss = total_train_loss / len(train_loader)
train_accuracy = accuracy_score(all_train_labels, all_train_preds)
train_losses.append(avg_train_loss)
train_accuracies.append(train_accuracy)

val_loss, val_metrics = evaluate(model, val_loader, criterion, device)
val_losses.append(val_loss)
val_accuracies.append(val_metrics['accuracy'])
scheduler.step()

if val_metrics['accuracy'] > best_val_accuracy:
    best_val_accuracy = val_metrics['accuracy']
    best_model_state = model.state_dict()
    epochs_no_improve = 0
else:
    epochs_no_improve += 1
    if epochs_no_improve >= patience:
        print("Early stopping triggered!")
        break

print(f"\nTrain loss: {avg_train_loss:.4f}, Train accuracy: {train_accuracy:.4f}")
print(f"Validation loss: {val_loss:.4f}, Validation metrics: {val_metrics}")

model.load_state_dict(best_model_state)
test_loss, test_metrics = evaluate(model, test_loader, criterion, device)
print(f"\nTest loss: {test_loss:.4f}, Test metrics: {test_metrics}")

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Loss Curves')

plt.subplot(1, 2, 2)
plt.plot(train_accuracies, label='Train Accuracy')
plt.plot(val_accuracies, label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Accuracy Curves')

plt.show()

```

✎ Let's see the difference between Hugging Face Trainer and PyTorch training!

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
set_seed(42)
train_model(model1, train_loader, val_loader, test_loader, device)
```

 Epoch 1/5: 100%
Evaluating: 100%

1441/1441 [06:18<00:00, 3.85it/s]
442/442 [00:36<00:00, 12.99it/s]

Train loss: 1.2027, Train accuracy: 0.5448
Validation loss: 1.0067, Validation metrics: {'accuracy': 0.627087461081234, 'precision': 0.646383693172929, 'roc_auc': 0.7353763
[40, 4, 29, 3, 5, 28, 2],
[114, 0, 483, 15, 75, 195, 136],
[44, 0, 56, 1529, 63, 65, 68],
[20, 0, 121, 76, 680, 285, 34],
[51, 0, 213, 34, 140, 684, 17],
[15, 0, 84, 29, 17, 15, 637]]})

 Epoch 2/5: 100%
Evaluating: 100%

1441/1441 [06:16<00:00, 3.85it/s]
442/442 [00:36<00:00, 13.02it/s]

Train loss: 0.9066, Train accuracy: 0.6622
Validation loss: 0.9099, Validation metrics: {'accuracy': 0.6645909991508633, 'precision': 0.674500951769136, 'roc_auc': 0.773145
[52, 35, 14, 1, 4, 5, 0],
[193, 2, 500, 21, 101, 128, 73],
[42, 1, 50, 1563, 93, 47, 29],
[107, 2, 74, 71, 837, 109, 16],
[159, 2, 178, 18, 241, 531, 10],
[30, 0, 112, 41, 26, 5, 583]]})

 Epoch 3/5: 100%
Evaluating: 100%

1441/1441 [06:17<00:00, 3.84it/s]
442/442 [00:36<00:00, 12.26it/s]

Train loss: 0.7461, Train accuracy: 0.7277
Validation loss: 0.8891, Validation metrics: {'accuracy': 0.6711010472686102, 'precision': 0.68603658381019, 'roc_auc': 0.7885824
[42, 53, 9, 1, 0, 6, 0],
[171, 1, 511, 17, 55, 195, 68],
[47, 0, 53, 1562, 70, 43, 50],
[182, 1, 55, 68, 679, 209, 22],
[189, 3, 111, 21, 124, 681, 10],
[38, 0, 100, 28, 13, 8, 610]]})

 Epoch 4/5: 100%
Evaluating: 100%

1441/1441 [06:17<00:00, 3.89it/s]
442/442 [00:36<00:00, 12.92it/s]

Train loss: 0.5856, Train accuracy: 0.7899
Validation loss: 0.8806, Validation metrics: {'accuracy': 0.6936031701103877, 'precision': 0.6948915934031019, 'roc_auc': 0.80385
[19, 64, 9, 1, 2, 12, 4],
[114, 2, 450, 34, 92, 222, 104],
[29, 0, 16, 1628, 61, 52, 39],
[36, 1, 36, 95, 843, 180, 25],
[60, 5, 104, 31, 189, 732, 18],
[15, 1, 50, 39, 18, 11, 663]]})

 Epoch 5/5: 100%
Evaluating: 100%

1441/1441 [06:17<00:00, 3.70it/s]
442/442 [00:36<00:00, 12.37it/s]

Train loss: 0.4130, Train accuracy: 0.8570
Validation loss: 0.9615, Validation metrics: {'accuracy': 0.6771865270308519, 'precision': 0.6840422191401141, 'roc_auc': 0.80753

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
set_seed(42)
train_model(model2, train_loader, val_loader, test_loader, device)
```



Epoch 1/5: 100%

721/721 [00:42<00:00, 13.65it/s]

/usr/local/lib/python3.10/dist-packages/torch/autograd/graph.py:744: UserWarning: Plan failed with a cudnnException: CUDNN_BACKEND
return Variable._execution_engine.run_backward(# Calls into the C++ engine to run the backward pass

Evaluating: 100%

221/221 [00:04<00:00, 51.63it/s]

Train loss: 1.8975, Train accuracy: 0.2232

Validation loss: 1.8003, Validation metrics: {'accuracy': 0.2796490234927823, 'precision': 0.25037696894890665, 'roc_auc': 0.5296}

```
[ 12,  0,  5,  66,  14,  13,  1],
[ 65,  0,  72, 578, 136, 108, 59],
[ 64,  0,  69, 1300, 178, 175, 39],
[ 56,  0,  66,  676, 256, 136, 26],
[ 72,  0,  43,  635, 161, 200, 28]
```